TASK 5: ENCAPSULATION & ABSTRACTION

Python OOP Practical Report — 30 Marks

1. Objective

To demonstrate controlled access to object data through encapsulation and use an abstract class with an abstract method implemented by a child class.

2. OOP Concept Used

Private attributes, controlled access through methods, abstract base class (ABC), @abstractmethod, and inheritance.

3. Source Code

```
from abc import ABC, abstractmethod


class BankAccount(ABC):
    def __init__(self, name, balance):
        self.name = name
        self.__balance = balance  # Private attribute

    def get_balance(self):
        return self.__balance

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            print("Amount deposited:", amount)
        else:
            print("Invalid deposit amount")

    @abstractmethod
    def account_type(self):
        pass


class SavingsAccount(BankAccount):
    def account_type(self):
        print("Account Type: Savings Account")

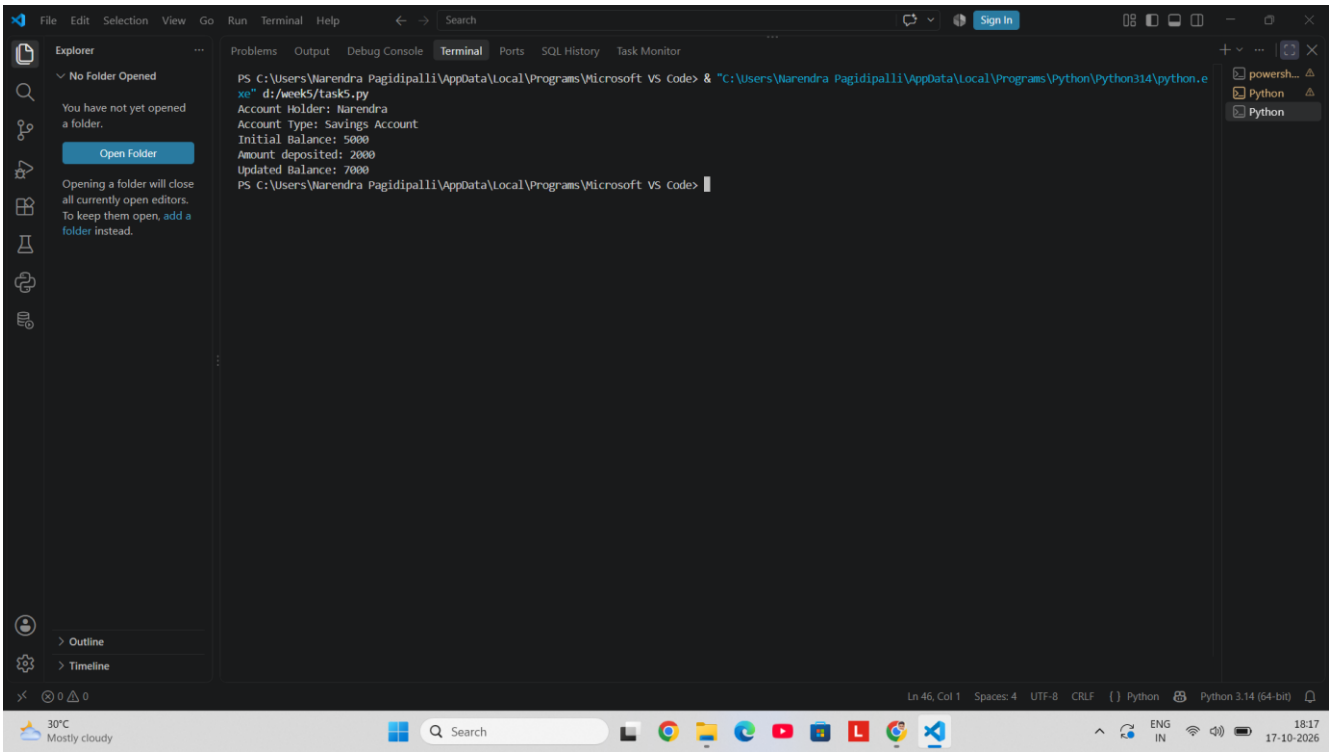

account = SavingsAccount("Narendra", 5000)

print("Account Holder:", account.name)
account.account_type()
print("Initial Balance:", account.get_balance())

account.deposit(2000)
```

```
print("Updated Balance:", account.get_balance())
```

4. Output



5. Explanation

- Encapsulation: __balance is a private attribute and is not normally accessed directly from outside the class.
- get_balance() provides controlled access to the balance.
- deposit() changes the balance only when the amount is positive.
- Abstraction: BankAccount inherits from ABC and declares account_type() as an abstract method.
- SavingsAccount implements account_type(), allowing its object to be created.

6. Learning Outcome

I learned how to protect data with encapsulation, validate changes through methods, and define an abstract interface that child classes must implement.

7. Conclusion

This program demonstrates encapsulation by controlling access to the account balance and abstraction through an abstract class and implemented method.